

# Claude Code — Model Routing Flows

## The Two Flows

```
Flow 1: Terminal → Claude Code → Anthropic API → Opus / Sonnet / Haiku
Flow 2: Terminal → Claude Code → Ollama          → Qwen (local model)
```

Both flows are correct. The difference is where the inference happens — Anthropic's cloud in Flow 1, your own machine in Flow 2.

## Flow 1 — Claude Code → Anthropic API

### How it works

Claude Code is a CLI tool (TypeScript, runs on Node/Bun) that wraps an agentic loop around Anthropic's `/v1/messages` API. When you run:

```
claude --model claude-opus-4-6
```

The `--model` flag is passed into every API request. Claude Code itself does no reasoning — it sends your prompt and tool results to Anthropic's servers, gets back a response, executes any tool calls (read file, run bash, etc.), and feeds the results back into the next API call.

### What is actually happening

```
You type a prompt
→ Claude Code CLI (QueryEngine)
  → POST api.anthropic.com/v1/messages
    { model: "claude-opus-4-6", messages: [...], tools: [...] }
  → Anthropic routes to Opus / Sonnet / Haiku
    ← Streams back response + tool_use blocks
  → Claude Code executes tools locally (reads files, runs bash)
  → Feeds tool_result back into next API call
  → Repeat until task is complete
```

### Model options

| Model             | Best for                             | Cost           |
|-------------------|--------------------------------------|----------------|
| claude-haiku-4-5  | Fast tasks, subagents, exploration   | Cheapest       |
| claude-sonnet-4-6 | Daily development, default workhorse | Mid            |
| claude-opus-4-6   | Complex architecture, hard debugging | Most expensive |

Switch during a session with `/model` , or set at launch:

```
claude --model claude-sonnet-4-6
```

## Requirements

- `ANTHROPIC_API_KEY` set in environment
- Active internet connection
- Anthropic account with API access or Claude subscription

---

## Flow 2 — Claude Code → Ollama → Local Model (Qwen)

### How it works

Ollama (v0.14+) added support for the Anthropic Messages API format. This means Claude Code can be redirected to Ollama by overriding three environment variables — Claude Code never knows it is talking to a local model instead of Anthropic's cloud.

```
export ANTHROPIC_AUTH_TOKEN="ollama"
export ANTHROPIC_API_KEY=""
export ANTHROPIC_BASE_URL="http://localhost:11434"

claude --model qwen2.5-coder:7b
```

Or using the newer Ollama launcher:

```
ollama launch claude --model qwen2.5-coder:latest
```

### What is actually happening

```
You type a prompt
→ Claude Code CLI (QueryEngine)
  → POST http://localhost:11434/v1/messages ← redirected by BASE_URL
    { model: "qwen2.5-coder:7b", messages: [...], tools: [...] }
  → Ollama translates Anthropic format → its own format
  → Runs inference locally (GPU/CPU)
  → Translates response back to Anthropic format
  ← Returns response to Claude Code
  → Claude Code executes tools locally
  → Repeat until task is complete
```

### Configuration

Save to `~/.claude/settings.json` to make it persistent:

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:11434",
    "ANTHROPIC_AUTH_TOKEN": "ollama",
    "ANTHROPIC_API_KEY": ""
  }
}
```

Recommended models for coding

| Model             | Size    | Tool calling | Context |
|-------------------|---------|--------------|---------|
| qwen2.5-coder:7b  | ~4.7 GB | Partial      | 32k     |
| qwen2.5-coder:14b | ~9 GB   | Partial      | 32k     |
| glm-4.7-flash     | ~21 GB  | Full         | 128k    |
| devstral-small    | ~14 GB  | Good         | 128k    |

**GLM-4.7 Flash is the most reliable for agentic tool use** — it supports structured tool calling which Claude Code depends on for file edits, bash execution, and project scanning.

Important limitation

Claude Code's agentic capabilities (file read/write, bash execution, project scanning) depend on tool calling. Under Ollama, whether these work depends entirely on the local model's tool-calling support. Models without reliable tool calling will behave as plain chat LLMs — they will respond but will not actually edit files or run commands.

Set context window to at least 20k tokens for Claude Code workflows:

```
OLLAMA_CONTEXT_LENGTH=20000 ollama serve
```

Requirements

- Ollama v0.14+ installed and running ( `ollama serve` )
- Enough RAM: 16 GB minimum, 32 GB recommended for 14B+ models
- No internet required once model is pulled
- No Anthropic API key required

Side-by-side Comparison

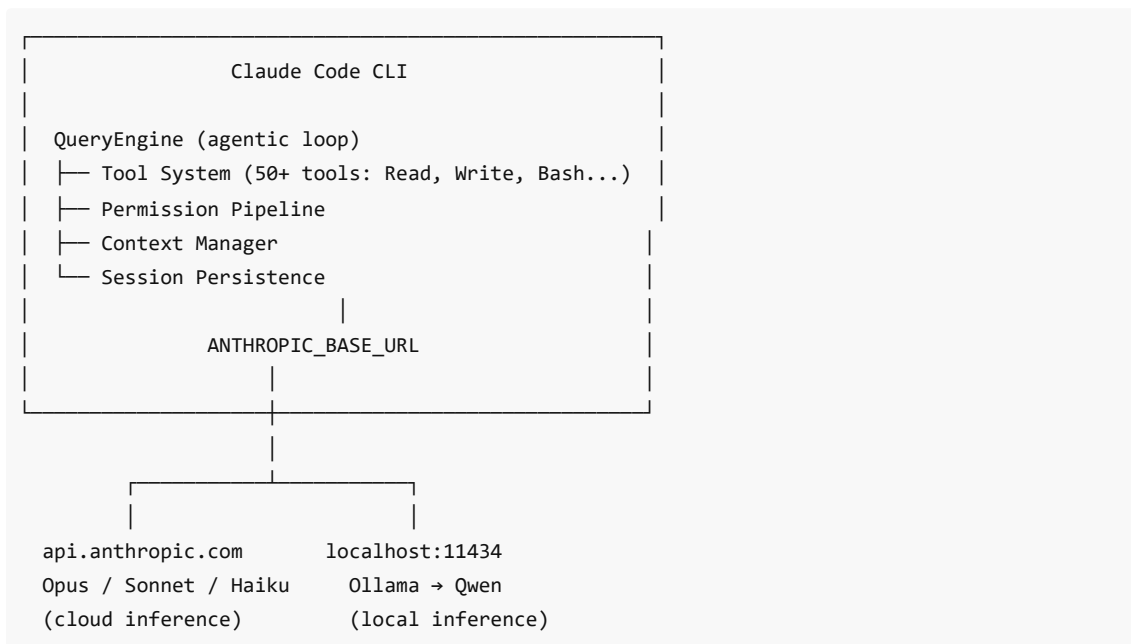
|                 | Flow 1 (Anthropic)      | Flow 2 (Ollama)                   |
|-----------------|-------------------------|-----------------------------------|
| Model quality   | Best (Opus/Sonnet)      | Good (depends on model)           |
| Cost            | API billed per token    | Free (electricity only)           |
| Privacy         | Code sent to Anthropic  | Fully local, never leaves machine |
| Speed           | Fast (cloud inference)  | Slower (depends on hardware)      |
| Tool calling    | Full agentic capability | Depends on local model            |
| Internet needed | Yes                     | No (after model pull)             |
| Setup effort    | Just an API key         | Ollama + model download           |

The Key Insight

Claude Code is not a model — it is an **agentic harness**. The CLI handles:

- The conversation loop
- Tool execution (file I/O, bash, git)
- Permission management
- Context compaction
- Session persistence

The model (whether Anthropic cloud or Ollama local) handles only the **reasoning** — deciding what to do next. Swapping the model via `--model` or `ANTHROPIC_BASE_URL` changes the intelligence quality of the loop. The harness itself stays identical.



## References

- Ollama × Claude Code integration: <https://docs.ollama.com/integrations/claude-code>
- Claude Code official docs: <https://code.claude.com/docs/en/overview>
- How Claude Code works: <https://code.claude.com/docs/en/how-claude-code-works>